In chapter 8 I said that there are two main ways to resolve services from the DI container for minimal API applications:

- Injecting services into an endpoint handler
- Accessing the DI container directly in Program.cs

You've seen the first of those approaches several times now in this chapter. In chapter 8 you saw that you can access services in Program.cs by calling `GetRequiredService<T>()` on `WebApplication.Services`:

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);
WebApplication app = builder.Build();
var settings = app.Services.GetRequiredService<EmailServerSettings>();
```

It's important, however, that you resolve only singleton services this way. The `IServiceProvider` exposed as `WebApplication.Services` is the root DI container for your app. Services resolved this way live for the lifetime of your app, which is fine for singleton services but typically isn't the behavior you want for scoped or transient services.

WARNING Don't resolve scoped or transient services directly from `WebApplication.Services`. This approach can lead to leaking of memory, as the objects are kept alive till the app exits and aren't garbage-collected.

Instead, you should only resolve scoped and transient services from an active scope. A new scope is created automatically for every HTTP request, but when you're resolving services from the DI container directly in Program.cs (or anywhere else that's outside the context of an HTTP request), you need to create (and dispose of) a scope manually.